

## **Projeto de Cálculo III - Pré definição de funcionalidades e necessidades do aplicativo**

**Grupo:** Anna Júlia, Bruna, Camila, Erick, Lara, Maurício

**Data:** 28/08

### **Funcionalidades do Aplicativo**

Aplicativo Android com java para calculadora visual de peças CAD 3D que permite ao usuário importar um objeto tridimensional e sua densidade, retornando: volume, massa, centro de massa, momentos e distribuição de massa. É pretendida uma interface simples para a entrada dos dados necessários, outputs e visualização do objeto em tempo real.

### **Responsabilidades Gerais**

- Parsing de STL binário
- Renderização 3D
- Sistema de partículas: gerar N partículas amostradas com probabilidade proporcional à densidade local (amostragem por rejeição dentro dos tetraedros) e reagir em tempo real ao slider de densidade.

### **Camada android/java:**

- Ciclo de vida e orquestração (MainActivity / Fragment)
- Gerenciar onCreate, onPause, onResume, onDestroy (além de pausar/retomar o GLSurfaceView corretamente para não vaziar contexto OpenGL)
- Conectar eventos de UI (clique no botão, slider) às classes de lógica, sem conter a lógica em si
- Repassar resultados prontos (CalculationResult) para exibição em TextView/UI.
- Abrir o seletor de arquivo via Storage Access Framework (Intent.ACTION\_OPEN\_DOCUMENT).
- Ler bytes do InputStream retornado pela Uri
- Delegar o parsing propriamente dito para MeshImporter (STL)
- VolumeCalculator, MassCalculator, Tetrahedron, DensityFunction (módulo Java isolado, ex.: Gradle module :math)

- Mesh3D, Vector3, Triangle, CalculationResult: estruturas simples (POJOs), sem lógica de renderização nem de Android.
- Parsing de malha grande e cálculo de tetraedros sem travar a UI thread
- Rodar cálculo pesado fora da main thread e devolver resultado via callback/runOnUiThread, para evitar travamentos (ex.: ExecutorService, AsyncTask ou Kotlin coroutines/Java CompletableFuture)
- Passar Mesh3D já processado e List<Particle> gerado para o MeshRenderer, sem a Activity conhecer detalhes de OpenGL (shaders, buffers).

### **Camada gráfica:**

- Conversão de dados para GPU (malha 3d em buffers)
- Shaders (vertex/fragment) para desenhar a malha com sombreado básico (ex.: Phong/Lambert).
- Projeção perspectiva + câmera (matrizes model/view/projection).
- Capturar gestos de toque (rotação por arrastar, pinça pra zoom) e traduzir em transformações da câmera
- Renderização de overlays
- Desenhar o marcador do centro de massa (uma esfera pequena ou eixo) sobre a malha.
- Desenhar as partículas do ParticleSystem como pontos/sprites, atualizando posição a cada frame.
- Loop de animação: onDrawFrame() chamado continuamente pelo GLSurfaceView.Renderer
- Sincronização com o ciclo de vida da Activity (pause/resume)

## **Conceitos a Serem Estudados e Aplicados**

### **1. Fundamentos matemáticos**

- Integrais duplas/triplas, mudança de ordem de integração
- Jacobiano e mudança de variáveis (cartesianas → cilíndricas/esféricas)
- Centro de massa, momentos estáticos e de inércia (fórmulas via integral tripla)

### **2. Representação geométrica**

- Estrutura de malha (vértices + faces triangulares)

- Parsing de STL binário
- Verificação de malha fechada (watertight), para volume ser bem definido

### **3. Cálculo numérico do volume/massa**

- Decomposição em tetraedros (triângulo da superfície + origem)
- Fórmula do volume assinado do tetraedro (produto misto /6)
- Soma com sinal → volume total (equivalente ao teorema da divergência)
- Extensão para massa com densidade constante ( $\rho \cdot V$ )

### **4. Densidade variável**

- Definir  $\rho(x,y,z)$  como função (ex: linear ao longo de um eixo)
- Quadratura numérica dentro de cada tetraedro (Gauss de baixa ordem)
- Massa = soma das integrais locais

### **5. Centro de massa e momentos**

- Centróide de cada tetraedro, ponderado por massa
- Soma vetorial → centro de massa global
- Momentos de inércia por fórmulas fechadas de tetraedro (evita nova integração numérica)

### **6. Interface básica**

- Import de arquivo (Storage Access Framework)
- Sliders/EditText para densidade (constante)
- Exibição de resultados: volume, massa, centro de massa

### **7. Visualização 3D**

- Setup OpenGL ES (GLSurfaceView) — renderizar a malha importada
- Câmera orbital (rotação por toque)
- Marcador visual do centro de massa

### **8. Densidade variável na UI**

- Sliders para parâmetros da função  $\rho(x,y,z)$
- Recalcular e atualizar resultados em tempo real

## **9. Sistema de partículas (opcional avançado)**

- Amostragem por rejeição dentro dos tetraedros, proporcional à densidade local
- Renderização das partículas como pontos/sprites no OpenGL
- Animação leve (jitter) para dar sensação de "distribuição viva"

## **10. Extras**

- Comparação numérica vs. analítica (formas primitivas: esfera, cilindro)
- Exportar relatório de resultados
- Múltiplos objetos na cena / composição de massas

Possível estrutura de classes no Java:

